Глава 12.
"Умные" указатели

МГТУ им. Н.Э. Баумана
Факультет Информатика и системы управления
Кафедра Компьютерные системы и сети
Лектор: д.т.н., проф.
Иванова Галина Сергеевна

12.1 Проблемы с динамическими объектами

- при использовании указателей ответственность за выделение памяти под объект и ее освобождение целиком лежит на программисте;
- на один и тот же объект может ссылаться несколько не связанных между собой указателей, следовательно возможно наличие указателей, ссылающихся на освобождённые или перемещённые объекты или попытки повторного освобождения памяти;
- нет никакой возможности проверить, указывает ли ненулевой указатель на корректные данные, либо «в никуда»;
- указатель на единичный объект и указатель на массив объектов в C++ никак не отличаются друг от друга.

Усложнение кода с учетом корректного освобождения памяти

Исходный код:

```
void proc()
{
    B *temp = new B;      // выделение памяти под объект
    temp->func();             // вызов метода для объекта
    delete temp;          // освобождение памяти
}
```

Код, освобождающий память при аварийном завершении:

```
void proc()
{    B *temp = NULL;
     try    {
          temp = new B;  // выделение памяти под объект
          B->func();     // вызов метода для объекта
     }
     catch (...)         // перехват всех исключений
     {                   // освобождение памяти при
          delete temp;   // аварийном завершении процедуры
          throw;         // возобновление исключения
     }
     delete temp;        // освобождение памяти при нормальном
                         // завершении процедуры
}
```

Проблема освобождения памяти при ошибке в конструкторе

```
class D
{
private:
    A * a;   B * b;
public:
    D() : a(new A), b(new B) { }
    ~D() {   delete a;   delete b; }
};
```

Деструктор вызывается только, если конструирование объекта завершено!

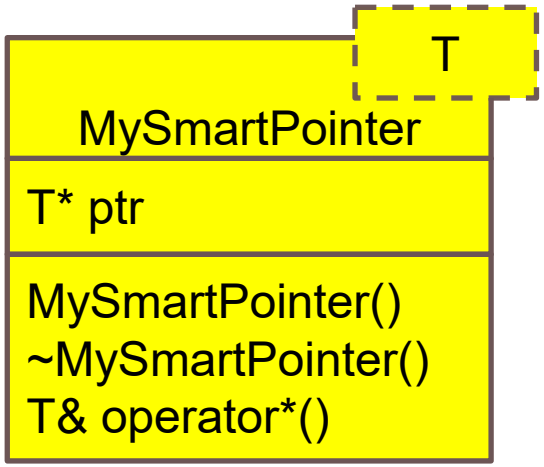
При ошибке в конструкторе часть памяти останется недоступной!!!

Технология *Resource Acquisition Is Initialization* (RAII) – «Захват ресурса есть инициализация».

Очевидное решение – вместо обычного указателя использовать объект-указатель, хранящий адрес и освобождающий память в своём деструкторе.

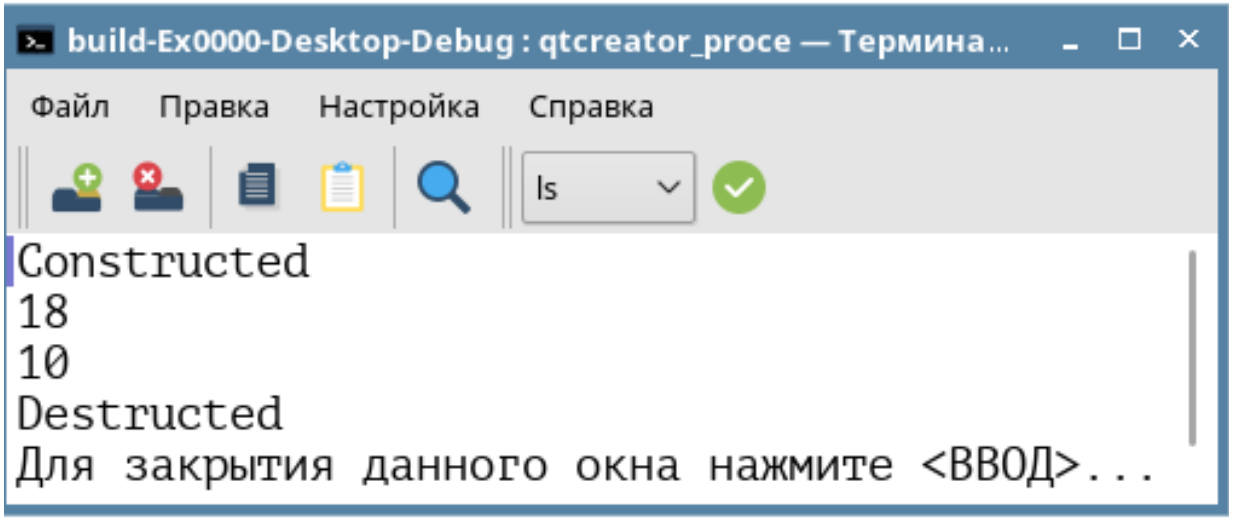
Пример Ex12_01

```
#include <iostream>
using namespace std;
template <typename T>
class MySmartPointer{
    private:    T* ptr;
    public:
        MySmartPointer(T* ptr) {
            this->ptr = ptr;
            cout << "Constructed\n";
        }
        ~MySmartPointer() {
            delete ptr;
            cout << "Destructed\n";
        }
        T& operator*() {
            return *ptr;
        }
};
```



Пример (продолжение)

```
int main()
{
    MySmartPointer<int> pointer(new int(18));
    cout << *pointer << endl;
    *pointer = 10;
    cout << *pointer << endl;
}
```



12.2 Шаблон `unique_ptr`

Шаблон реализует семантику «владения», при которой всегда существует только один объект-указатель, хранящий адрес выделенного фрагмента динамической памяти.

В отличие от ранее предлагаемого шаблона **`auto_ptr`** реализация шаблона **`unique_ptr`** заменяет *копирование* и *присваивание перемещением*, при котором копируемый адрес уничтожается после переписи в новый объект-указатель. Таким образом становится невозможным наличие двух и более объектов-указателей, адресующих один объект, что исключает ошибки, связанные с повторным освобождением памяти объекта.

Основные методы `unique_ptr`

- конструкторы и деструктор;
- **`get()`** – возвращает хранимый адрес;
- **`release()`** – возвращает хранимый адрес и записывает вместо него **`nullptr`** в объект-указатель;
- **`reset()`** – освобождает память и, если параметр – новый динамический объект указан, то записывает в объект новый адрес, если параметр не указан, то адрес устанавливается в **`nullptr`**;
- **`move()`** – корректно меняет владельца фрагмента памяти;
- **`operator*()`** – используется для доступа к объекту по адресу;
- **`operator->()`** – используется для доступа к полям объекта по адресу;
- **`operator bool()`** – преобразует значение в **`bool`** для проверки выделения памяти и др.

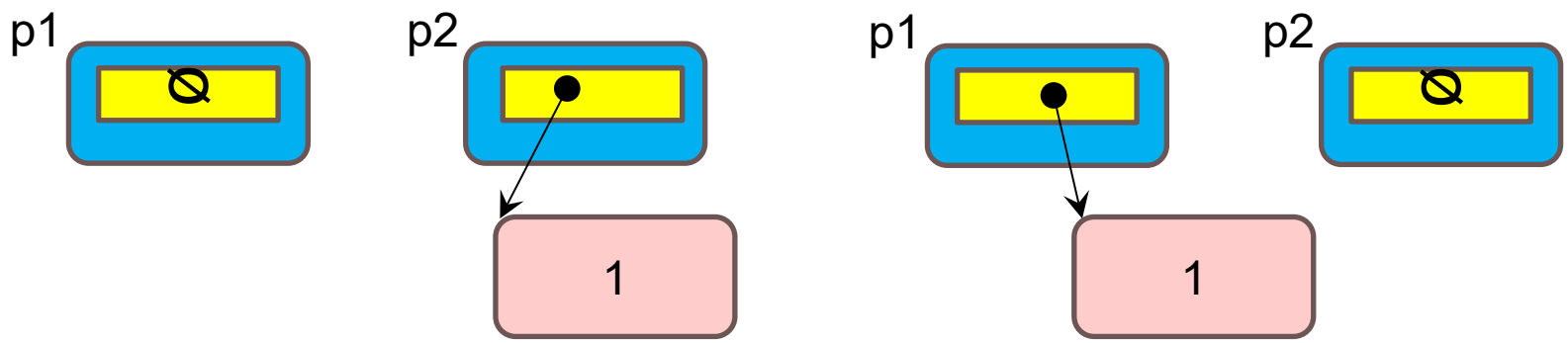
Шаблон функции: **`make_ unique()`** – (используется с версии C++14) выделяет память сразу под программный объект и объект-указатель.

Пример Ex12.02. Создание объекта-указателя и работа с ним

```
#include <iostream>
#include <memory>
using namespace std;
class A
{
public:
   int x;
   A(int X):x(X){cout<<Constructor"<<endl;}
   A(){}
   ~A(){cout<<"Destructor"<<endl;}
};
```

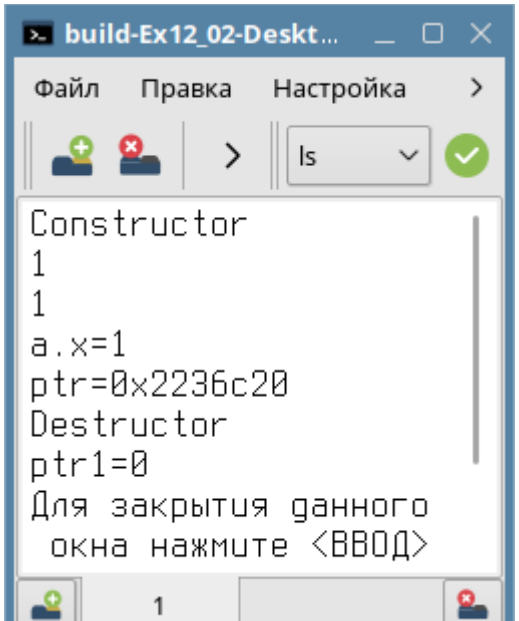
Создание объекта-указателя и работа с ним(2)

```
int main(){
   unique_ptr<A> p1;          // неинициализированный объект
   //unique_ptr<A> p2(new A(1)); // инициализированный объект
   unique_ptr<A> p2 = make_unique<A>(1);    // C++14
   // unique_ptr<A> p3(p2); !!! нельзя!!!
   // p1 = p2;              !!! нельзя!!!
    p1=move(p2);           // смена владельца объекта
```



Создание объекта-указателя и работа с ним(3)

```
  cout << p1->x << endl;      // обращение к полю через указатель
  cout << (*p1).x << endl;    // обращение к полю через ссылку
  A *ptr = p1.get();          // указатель на хранимый объект
  cout << "ptr=" << ptr << endl;
  A &a= *p1;                  // ссылка на хранимый объект
  cout << "a.x=" << a.x << endl;
  if (p1) p1.reset();         // проверка и освобождение памяти
  A *ptr1 = p1.get();         // указатель на хранимый объект
  cout << "ptr1=" << ptr1 << endl;
  return 0;
}
```



Пример Ex12.03. Создание указателя для массива

```
#include <iostream>
#include <memory>
  using namespace std;
class A {
  public:
     int x;
     A(int X):x(X){}
     A(){}
     ~A(){cout<<"Destructor "<<x<<endl;}
};
int main() {
   int n;
   std::unique_ptr<A[]> arr(new A[2]);
   for(size_t i=0;i<2;i++){
      cout<<"Enter n:"; cin>>n; arr[i].x = n;
   }
   return 0; }
```

Для создания указателя динамического массива объектов следует, объявляя его, добавить квадратные скобки

Enter n:3
Enter n:-56
Destructor -56
Destructor 3

Ограничение

Невозможность создания копий адреса не позволяет использовать указатели, построенные по шаблонам **`unique(auto)_ptr`**, для создания списковых структур, поскольку в процессе обработки список будет разрушаться. Выходом из этой ситуации является использование специальных "умных" указателей:

а) **`shared_ptr`** – создает совладельцев с подсчётом ссылок;

б) **`weak_ptr`** – создает "слабые" указатели, не влияющие на количество ссылок.

12.3 Шаблоны `shared_ptr` и `weak_ptr`

Шаблон **`shared_ptr`** используется для создания объектов-указателей, совместно владеющих выделенным фрагментом памяти. Количество объектов-указателей, владеющих одним и тем же фрагментом памяти, хранится и корректируется при создании и уничтожении объектов-указателей. Если это количество обнуляется, то память автоматически освобождается.

Шаблон **`weak_ptr`** для создания объектов-указателей, содержащих копию адреса, но не владеющих фрагментом памяти и, следовательно, при создании и уничтожении объектов-указателей не влияющих на счетчик объектов – владельцев фрагмента памяти. Позволяет реализовать циклические ссылки. Однако через указатель типа **`weak_ptr`** невозможно обратиться к полям объекта.

Основные методы shared_ptr

- конструкторы и деструктор;
- **`swap()`** – меняет местами два адреса;
- **`reset()`** – освобождает память и, если параметр – новый динамический объект указан, то записывает в объект новый адрес, если параметр не указан, то адрес устанавливается в **`nullptr;`**
- **`get()`** – возвращает хранимый адрес;
- **`use_count()`** – возвращает количество владельцев фрагмента памяти;
- **`operator=()`** – позволяет присвоить значение другому объекту;
- **`operator*()`** – используется для доступа к объекту по адресу;
- **`operator->()`** – используется для доступа к полям объекта по адресу;
- **`operator bool()`** – преобразует значение в **`bool`** для проверки выделения памяти и др.

Шаблон функции: **`make_shared <T>()`** – (с версии C++11) выделяет память сразу под программный объект и объект-указатель.

Основные методы wear_ptr

- конструкторы и деструктор;
- **`swap()`** – меняет местами два адреса;
- **`reset()`** **-** – освобождает память и, если параметр – новый динамический объект указан, то записывает в объект новый адрес, если параметр не указан, то адрес устанавливается в **`nullptr;`**
- **`lock()`** – создает временную копию указателя типа shared_ptr для организации доступа к полям и методам динамического объекта;
- **`expired()`** – проверяет жив ли еще объект, на который указывает объект-указатель, так как соответствующая память может быть уже освобождена;
- **`use_count()`** **-** возвращает количество владельцев фрагмента памяти;
- **`operator=()`**– позволяет переприсвоить значение;
- **`operator bool()`** **–** позволяет проверить ненулевой адрес динамического объекта.

Сравнение методов share_ptr и wear_ptr

Методы share_ptr:

- конструкторы и деструктор;
- `swap()`
- `reset()`
- `use_count()`
- `operator=()`
- `operator bool()`
- `get()`
- `operator*()`
- `operator->()`

Методы wear_ptr:

- конструкторы и деструктор;
- `swap()`
- `reset()`
- `use_count()`
- `operator=()`
- `operator bool()`
- `lock()`
- `expired()`

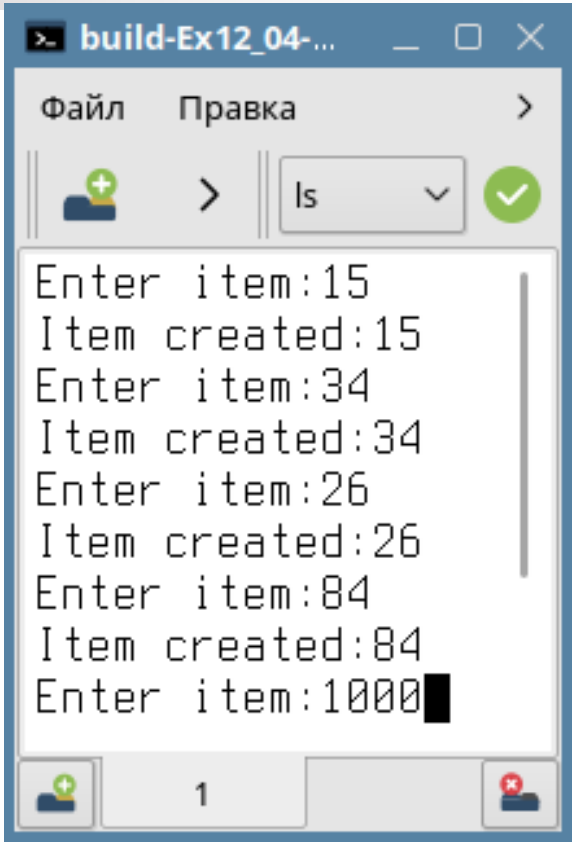
Пример Ex12.04 Список с "умными" указателями

```
#include <iostream>
#include <memory>
  using namespace std;
class Element{
  public:
     int num;
     shared_ptr <Element> p;
     Element(int Num)/*:p(nullptr)*/{
        num=Num; cout<<"Item created:"<<num<<endl;
      }
     ~Element(){ cout<<"Item released:"<<num<<endl; }
};
  int main() {
    int n;
    shared_ptr <Element> first;  // будет создан
                                 // объект-указатель с nullptr
    weak_ptr <Element> q;
```

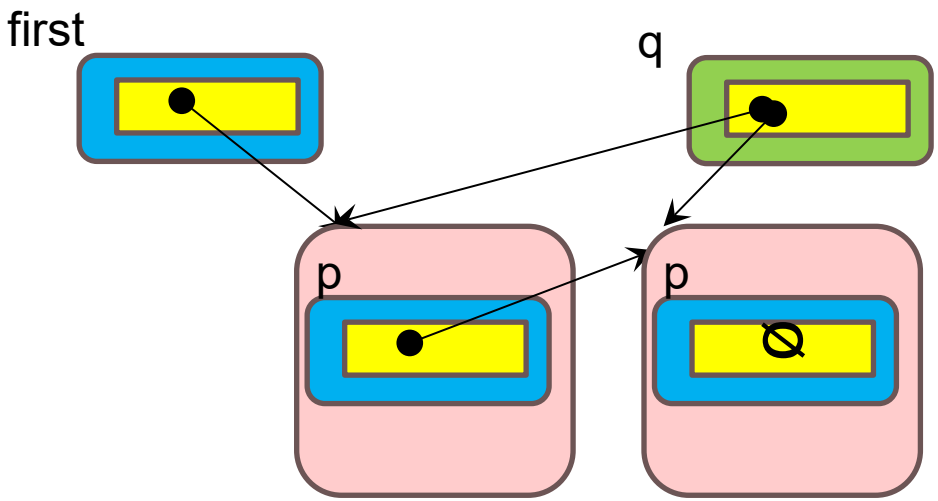
Можно тоже shared_ptr !

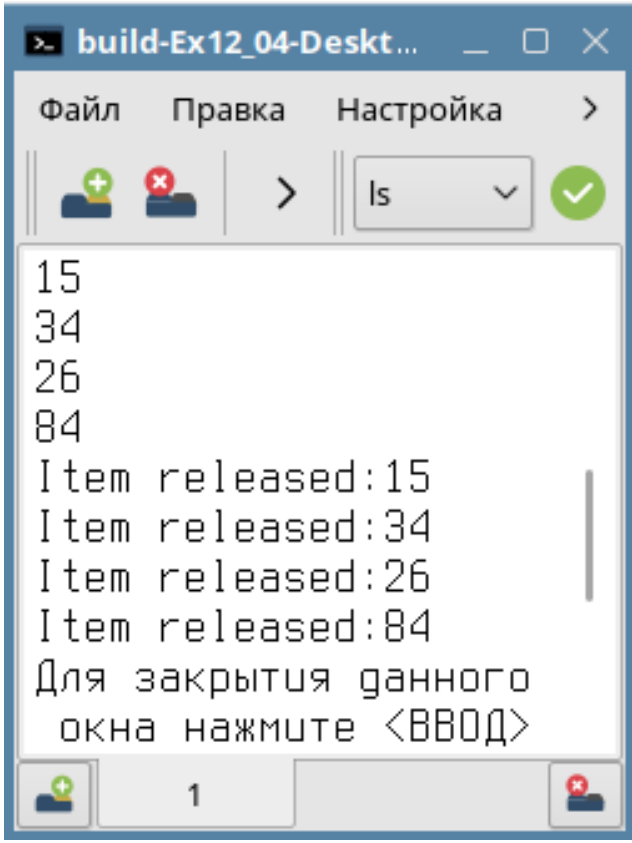
Создание списка

```
cout << "Enter item:";
cin >> n;
if (n !=1000){
    first = make_shared<element>(n);
    q=first;
  }
cout << "Enter item:";
cin >> n;
while (n !=1000){
    q=(q.lock())->p=make_shared<element>(n);
    cout << "Enter item:";
    cin >> n;
 }
```



Процесс создания списка



Вывод содержимого списка на экран

```
  if (!first)
     cout<<"List is empty.";
  else {
    q=first;
    while(q.lock()){
       cout<<q.lock()->num<<endl;
       q=(q.lock())->p;
    }
  }
  return 0;
}
```